use `WebApplicationFactory` to simulate requests to your application and verify that it generates the correct response.

In the final section of this chapter I'll demonstrate how to use the SQLite database provider for Entity Framework Core (EF Core) with an in-memory database. You can use this provider to test services that depend on an EF Core `DbContext` without having to use a real database. That prevents the pain of having unknown database infrastructure and resetting the database between tests, with different people having slightly different database configurations.

In chapter 35 I showed how to write unit tests for an exchange-rate calculator service, such as you might find in your application's domain model. If well designed, domain services are normally relatively easy to unit-test. But domain services only make up a portion of your application. It can also be useful to test your ASP.NET Core-specific constructs, such as custom middleware, as you'll see in the next section.

36.1 Unit testing custom middleware

In this section you'll learn how to test custom middleware in isolation. You'll see how to test whether your middleware handled a request or whether it called the next middleware in the pipeline. You'll also see how to read the response stream for your middleware.

In chapter 31 you saw how to create custom middleware and encapsulate middleware as a class with an `Invoke` function. In this section you'll create unit tests for a simple health-check middleware component, similar to the one in chapter 31. This is a basic implementation, but it demonstrates the approach you can take for more complex middleware components.